Operational Analysis

Description of a Queueing System

- a simple queue consists of a buffer/queue where arriving jobs wait to be served by a server – after completion, they depart
- Must describe:

1. Arrivals: the time between arrivals is given by a probability distribution
2. Queue – size (infinite or finite)
3. How server operates – number of servers, order of service (FCFS, LCFS, etc.), processing times (the probability distribution that determines them)

Performance Indicators/Metrics

1. Response time – time from arrival to departure
2. Throughput – number of jobs served per unit time

Queueing Networks

We may be interested in more complicated systems, where queues are connected together.

Analysis Directions

We are faced with two possibilities when doing analysis:

1. Exact analysis: try to formulate equations that yield the desired performance metric, then solve
2. Simulation: by generating realizations/samples from the underlying distributions, we can follow the logic of the system to estimate the performance measure(s) of interest. For example, generate a (large) number of “real” response times, compute average

Operational Laws

First cut at analytic models. Suppose that we have the following data:

- $A_i(t)$: number of arrivals to device/node i at time t
- $C_i(t)$: number of completions (departures) from node i at time t
- $B_i(t)$: busy time of node i at time t

Some Simple Calculations

From the data, we can quickly derive the following quantities:

- Arrival rate at node i: $\lambda_i(t) = A_i(t)/t$
- Throughput of node i: $X_i(t) = C_i(t)/t$
- Utilization of node i: $\rho_i(t) = B_i(t)/t$
- Average processing time at node i: $S_i(t) = B_i(t)/C_i(t)$

Ergodicity Assumptions

Assume that the system is *ergodic*, i.e., time averages lead to actual underlying means:

$$\lambda_i(t) \to \lambda_i$$
$$X_i(t) \to X_i$$
$$\rho_i(t) \to \rho_i$$
$$S_i(t) \to E[S_i] = 1/\mu_i$$

Utilization Law

$$\frac{B_i(t)}{t} = \frac{C_i(t)}{t} \cdot \frac{B_i(t)}{C_i(t)}$$

$$\rho_i(t) = X_i(t) S_i(t)$$

Letting $t \to \infty$,

$$\rho_i = X_i E[S_i]$$

Forced Flow Law

Suppose that $A_i(t) = C_i(t)$. Let $E[V_i]$ be the expected number of visits to node i per job. This requires a reference node, which we will call node 0. By definition, $E[V_0] = 1$. So, we have

$$\lim_{t\to\infty} \frac{C_i(t)}{t} = \lim_{t\to\infty} \frac{C_0(t)}{t} E[V_i]$$

$$E[V_i] = \lim_{t\to\infty} \frac{C_i(t)}{C_0(t)}$$

Forced Flow Law

Define: system throughput X measured through node 0. The throughput of node i is

$$X_i(t) = \frac{C_i(t)}{t} = \frac{C_i(t)}{C_0(t)} \frac{C_0(t)}{t}$$

Taking limits as $t \to \infty$

$$X_i = E[V_i]X$$

Device Demands

- Utilization Law: $\rho_i = X_i E[S_i]$
- Forced Flow Law: $X_i = E[V_i]X$
- Combining these two: $\rho_i = E[S_i]E[V_i]X$
- Define the average demand at node i: $E[D_i] = E[S_i]E[V_i]$
- Then, $\rho_i = E[D_i]X$

The *demand* at node i, $E[D_i]$, is the expected processing time at node i summed over *all* visits of a job. We will see that the *bottleneck* is the device with the highest $E[D_i]$.

Little's Law

$$E[N_i] = \lambda_i E[T_i]$$

This relates the number of jobs in a queueing system, $E[N_i]$, the arrival rate to the queueing system, λ_i, and the mean response time, $E[T_i]$.

Outline of Derivation

- The number in system at time t, $N(t)$, can be written as $N(t) = A(t) - C(t)$
- The arrival rate at time t is $\lambda(t) = A(t)/t$
- The average response time in system in system for jobs that arrived in $(0, t)$ is $T(t) = \frac{1}{A(t)} \int_0^t \big(A(u) - C(u)\big) du$
- The average number in system at time t is $\overline{N} = \frac{1}{t} \int_0^t \big(A(u) - C(u)\big) du$

Outline of Derivation

So,

$$\bar{N}(t) = \frac{\int_0^t \big(A(u) - C(u)\big)du}{t} = \frac{T(t)A(t)}{t} = \lambda(t)T(t)$$

Letting $t \to \infty$,

$$E[N] = \lambda E[T]$$

Users/Clients

We consider a model where there are M users (clients) who operate as follows:

1. Each user thinks for a period of time that has mean $E[Z]$ time units (the think time)
2. At the end of a think time, the user submits the job/request to a subsystem
3. Once processing is complete at the subsystem (and returned to the user), the user begins another think time

General Response Time Law

Consider a system with M users connected to a subsystem that has K nodes. We are interested in the mean response time for the subsystem, $E[T]$. Inside the subsystem the mean number of jobs $E[N]$ satisfies

$$
\begin{aligned}
E[N] &= XE[T] \\
&= E[N_1] + E[N_2] + \cdots + E[N_K] \\
XE[T] &= X_1E[T_1] + X_2E[T_2] + \cdots + X_KE[T_K] \\
E[T] &= E[V_1]E[T_1] + E[T_2]E[V_2] + \cdots + E[V_K]E[T_K]
\end{aligned}
$$

General Response Time Law

This then gives our final result

$$E[T] = \sum_{i=1}^{K} E[V_i]E[T_i]$$

Interactive Response Time Law

- Consider the same system as for the General Response Time Law
- The total mean cycle time of a client's job is $E[Z] + E[T]$
- So, in a given time period of length t, each client generates on average

$$\frac{t}{E[Z] + E[T]}$$

requests

Interactive Response Time Law

- The throughput is then given by

$$X = \frac{M\left(\frac{t}{E[Z] + E[T]}\right)}{t} = \frac{M}{E[Z] + E[T]}$$

- The end result is

$$E[T] = \frac{M}{X} - E[Z]$$

Bottleneck Analysis

Let $D = E[D_1] + E[D_2] + \cdots + E[D_K]$ be the average total demand. We then have

$$X \leq \min\left(\frac{1}{D_{max}}, \frac{M}{D + E[Z]}\right)$$
$$E[T] \geq \max(D, MD_{max} - E[Z])$$

where $D_{max} = \max_i E[D_i]$ is the maximum demand in the subsystem. D_{max} is the "bottleneck demand" and the device that achieves D_{max} is the "bottleneck".

Proof of Bounds

1. At the bottleneck,

$$\rho_{max} = XD_{max} \leq 1 \Longrightarrow X \leq \frac{1}{D_{max}}$$

2. With one client,

$$\begin{aligned} E[T] &= E[D_1] + E[D_2] + \cdots + E[D_K] \\ &= D \\ E[T] &\geq D \end{aligned}$$

as the mean response time is lowest with one client.

Proof of Bounds

From the Interactive Response Time Law,

3.

$$E[T] = \frac{M}{X} - E[Z]$$
$$\geq MD_{max} - E[Z]$$

4.

$$X = \frac{M}{E[T] + E[Z]}$$
$$\leq \frac{M}{D + E[Z]}$$

Example

A computer system has one CPU and two disks. After monitoring the system for two hours, the following observations were made. The utilization of the CPU was 43 percent and the utilization of the first disk was 66 percent. Each transaction to the system makes 5 I/O requests to the first disk and 6 to the second disk. The average processing time (per request) at the two disks were both 20 msec.

a) What was the throughput of the system (in transactions per second)?

b) What was the utilization of the second disk?

c) What was the total processing demand (in milliseconds per transaction) at the CPU?

Another Example

Suppose that we have M clients that submit requests to three single-server nodes in series (i.e., a client request needs to be processed by three servers, in order). The mean demands at the three nodes are $E[D_a] = 2$, $E[D_b] = 3$, $E[D_c] = 1$. The average think time of a client, $E[Z]$, is 30.

Bottleneck Analysis

The throughput is bounded above by

$$\min\left(\frac{1}{3}, \frac{M}{36}\right)$$

and the mean response time is bounded below by

$$\max(6, 3M - 30)$$

One can see that these bounds both have break points at $M^* = 12$. This value is often known as the number of clients that the system can support. Below this number, performance scales well, above this number, performance degrades considerably.

Load Balancing

Suppose that we were free to choose $E[D_a]$, $E[D_b]$ and $E[D_c]$ to be any (nonnegative) value. What would be a good choice?

One More Example

A database server receives requests from 50 clients. Each request to the DB server requires the five reads be made on average from the server's single disk. The average read time is 9 msec. Each database request requires on average 15 msec of CPU time to be processed. Consider the following three proposed design changes:

i. The average number of disk reads per access can be reduced from 5 to 2.5

ii. The disk is replaced by one 60 percent faster

iii. The CPU speed is doubled

If you were to do one of these, which would you do? If you were to do two of them?

Final Example

- Application with three microservices: 1, 2 and 3.
- Users have a think time of 1 minute
- How many users can the system support?

Final Example

1. Need to estimate processing rates, in this case throughput was easy to measure

$$\rho_i = X_i E[S_i]$$

So, load each microservice until utilization close to 1, then $E[S_i] \approx \frac{1}{X_i}$.

2. Get visit ratios from architecture, in this case, 2, 1, 1

3. Here, $S_1 = 1/170$, $S_2 = 1/450$, $S_3 = 1/250$. Visit ratios are 2, 1, 1. So demands are 2/170, 1/450, 1/250

Final Example

4.

$$\frac{M^*}{D + E[Z]} = \frac{1}{D_{max}}$$

Here, $D + E[Z] \approx E[Z]$, so $M^* \approx 170 \times 60 \approx 5000$